Типы данных и управление потоком

ПЕРЕМЕННЫЕ, КОНСТАНТЫ,
IF, FOR И SWITCH



X5 Tech

Какую задачу решает эта тема?

- Типы и ветвления для логики сервиса заказов
- Данные → условие → результат

Главная цель: научиться реализовывать небольшие алгоритмы

ЧТО ДАЁТ НАВЫК



Выбирать подходящий тип

Различать int, float64, string, bool, byte и rune



Объявлять данные

Использовать var, :=, const и zero values



Управлять выполнением

Применять if, for, range и switch

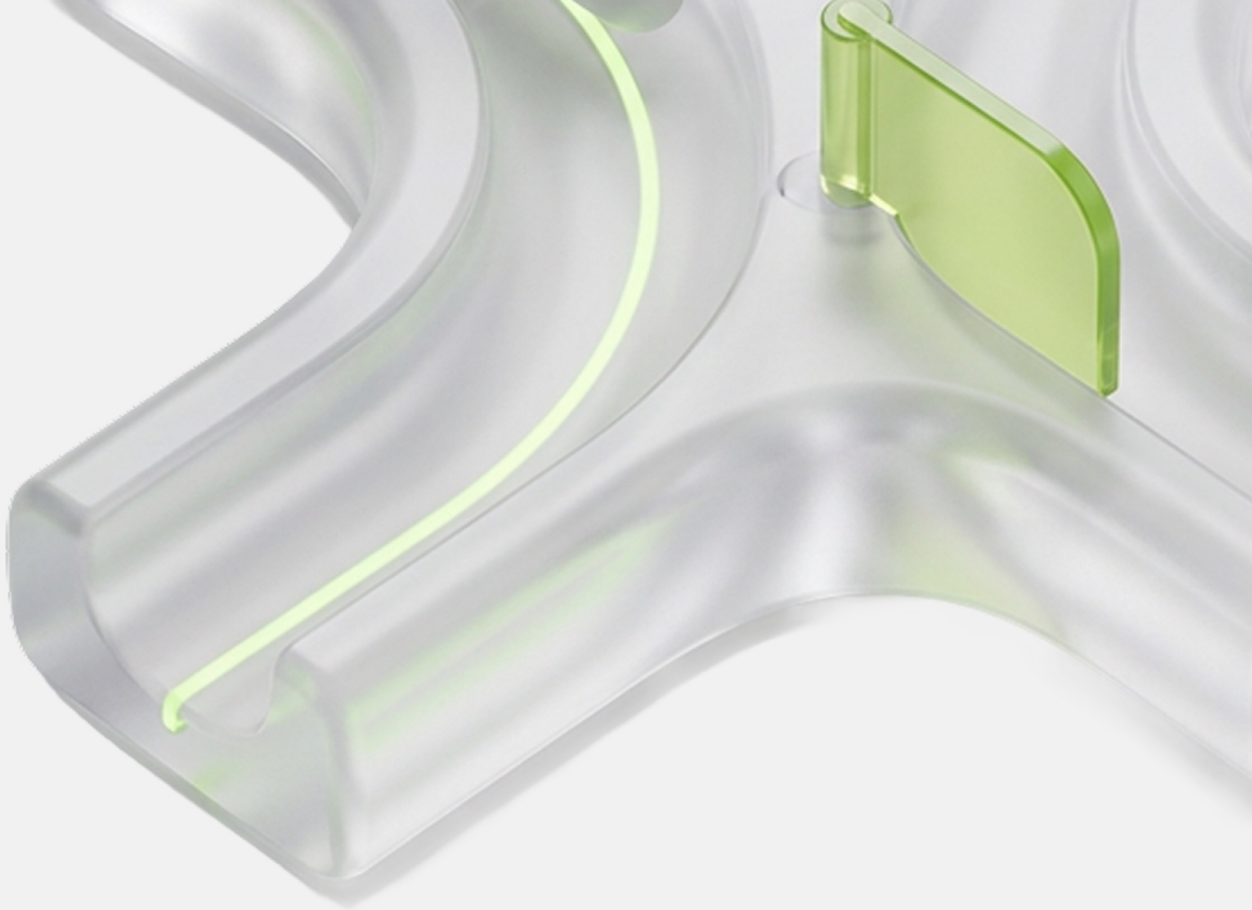


Учитывать границы

Замечать переполнение, точность и пустые значения

Поговорим о том, что уже прошли

1. Умеем создавать модуль
2. Умеем запускать программу
3. Умеем запускать тесты
4. Умеем форматировать код



СЛЕДУЮЩИЙ ШАГ

Типы и управление

Научимся хранить значения, сравнивать их и повторять действия без лишней сложности

2 МИНУТЫ НА ОБСУЖДЕНИЕ

Какой тип выбрать для цены заказа?

СРАВНИТЕ ВАРИАНТЫ

float64

int

int64

Как выбор повлияет на точность и диапазон?

ОТВЕТ

Деньги удобно хранить в целых мин. единицах

Таким образом цена не потеряет точности

```go
priceKopecks := int64(19990)
quantity := int64(3)

total := priceKopecks * quantity
fmt.Println(total) // 59970
```

СРАВНИВАЕМ:

Тип:	int64
Единица:	Копейки
Итог	59970

Тип определяет допустимые операции

Компилятор не смешивает разные числовые типы автоматически

ЯВНЫЕ ПРЕОБРАЗОВАНИЯ:

```
var count int = 3
var price float64 = 19.9

// count * price – ошибка

total := float64(count) * price
fmt.Println(total)
```

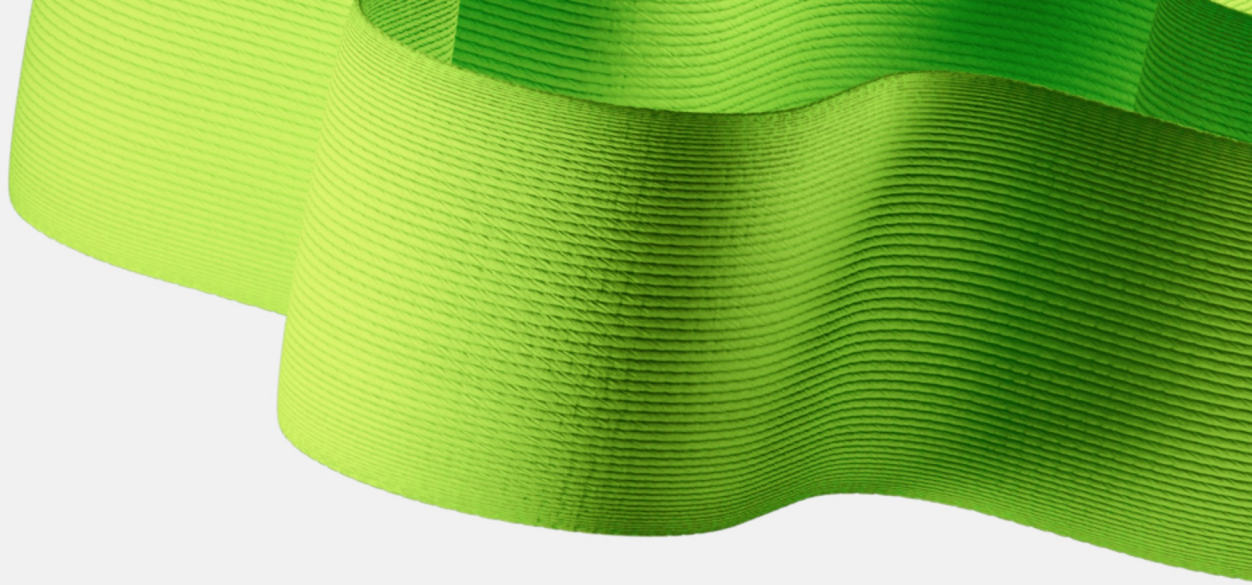
Строгая типизация делает границы явными

ПОЧЕМУ ЭТО ПОЛЕЗНО?

1. Преобразование видно в коде
2. Потеря точности не происходит молча
3. Намерение автора можно проверить

Ошибки выбора типа

Неверный диапазон или представление данных создают баги на границах



ЧТО ИДЁТ НЕ ТАК:

- ✖ int8 переполняется
- ✖ float сравнивают через ==
- ✖ byte путают с rune
- ✖ Преобразование теряет данные

КЛЮЧЕВОЙ ВЫВОД

Тип влияет на поведение программы. Его выбирают под данные, а не по привычке

Выбираем тип осознанно

Что дают базовые типы

Каждый тип ограничивает значения и доступные операции

ЧИСЛОВЫЕ:

- Int / int64
- float64
- byte / rune
- Явные преобразования

ОСТАЛЬНЫЕ:

- string
- bool
- zero values
- Составные выражения
- Тип задаёт правила

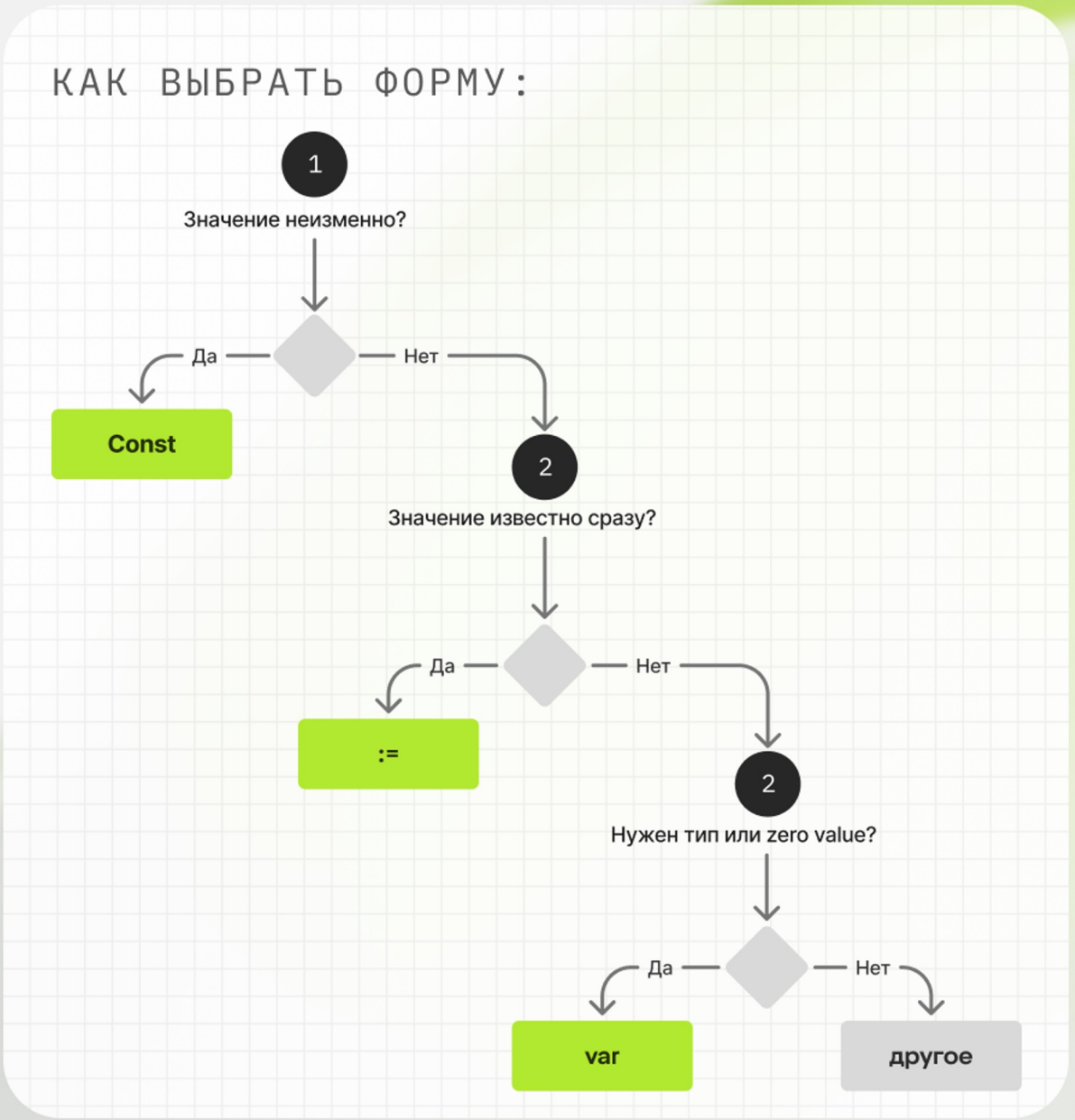
Дайте значению тип, имя и понятную область видимости

- **var** – явное объявление
- **:=** – короткая форма внутри функции
- **const** – значение, которое не меняется

Явное
лучше
СлучАйНого

Как объявлять значения

Каждый тип ограничивает значения
и доступные операции



ОПРЕДЕЛЕНИЕ

«Переменная» – это…

Именованное значение определённого типа,
которое может изменяться.

Базовые объявления

Go выводит тип там, где это однозначно

СРАВНИВАЕМ:

```
var retries int
name := "order-service"
const maxRetries = 3

fmt.Println(retries) // 0
fmt.Println(name)
fmt.Println(maxRetries)
```

ОБЪЯСНЕНИЕ ОБЪЯСНЕНИЕ

СЕЙЧАС ВАЖНЫ ТРИ ФОРМЫ

Zero value делает переменную предсказуемой даже до явной инициализации

Управление потоком

Большинство алгоритмов
строятся из трёх понятных шагов

1. **IF**
   Выбираем ветку

```
if total > limit
```

2. **FOR**
   Повторяем действия

```
for _, v := range values
```

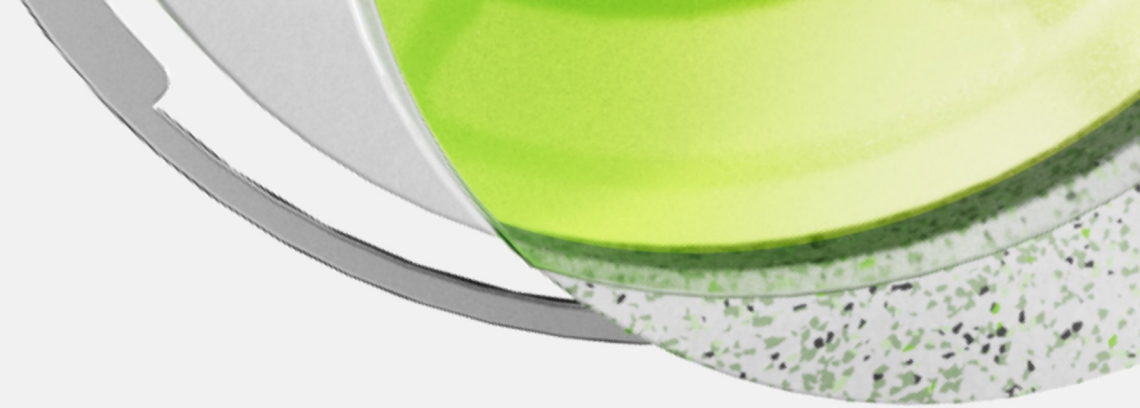
3. **SWITCH**
   Разбираем случаи

```
switch status
```



Один цикл – три формы

for покрывает счётчик, условие
и перебор коллекции



ИСХОДНЫЙ КОД:

```
for i := 0; i < 3; i++ { }

for ready == false { }

for index, value := range items { }
```

ОБЪЯСНЕНИЕ

Форма цикла должна подчёркивать намерение

1. **Классическая форма** – счётчик и шаг
2. **While-стиль** – только условие
3. **range** – индекс и значение

Ветвление через switch

switch читается сверху вниз
и не требует обязательного break

ИСХОДНЫЙ КОД:

```
status := "paid“

switch status {
case "new":
    fmt.Println("создан")
case "paid", "shipped":
    fmt.Println("в работе")
default:
    fmt.Println("неизвестен")}
```

ОБЪЯСНЕНИЕ

Switch выбирает ветку без обязательного break

1. Проверяем случай
2. Выбираем ветку
3. Не проваливаемся ниже

Когда алгоритм ломается

Граничный случай часто показывает
неверное предположение о данных

ИСХОДНЫЙ КОД:

```
values := []int{}

sum := 0
for _, v := range values {
    sum += v
}

avg := sum / len(values) // panic
```

Что ожидали:

Среднее число

Что получили:

Объявлять данные

Какой случай:

Пустой срез



До вычисления проверяем len(values) == 0

Типы и управление работают вместе

Хороший алгоритм начинается
с модели данных

ИЗ ЧЕГО СОСТОИТ ХОРОШИЙ АЛГОРИТМ:

1. Подходящий тип
2. Понятные ветки

Типы ограничивают данные

Конструкции управления выражают правила

Граничные случаи становятся видимыми в коде

Путь алгоритма

От входных данных
к проверенному результату

1. Выбрать тип
2. Объявить данные
3. Проверить условие
4. Пройти циклом
5. Разобрать switch
6. Обработать границы



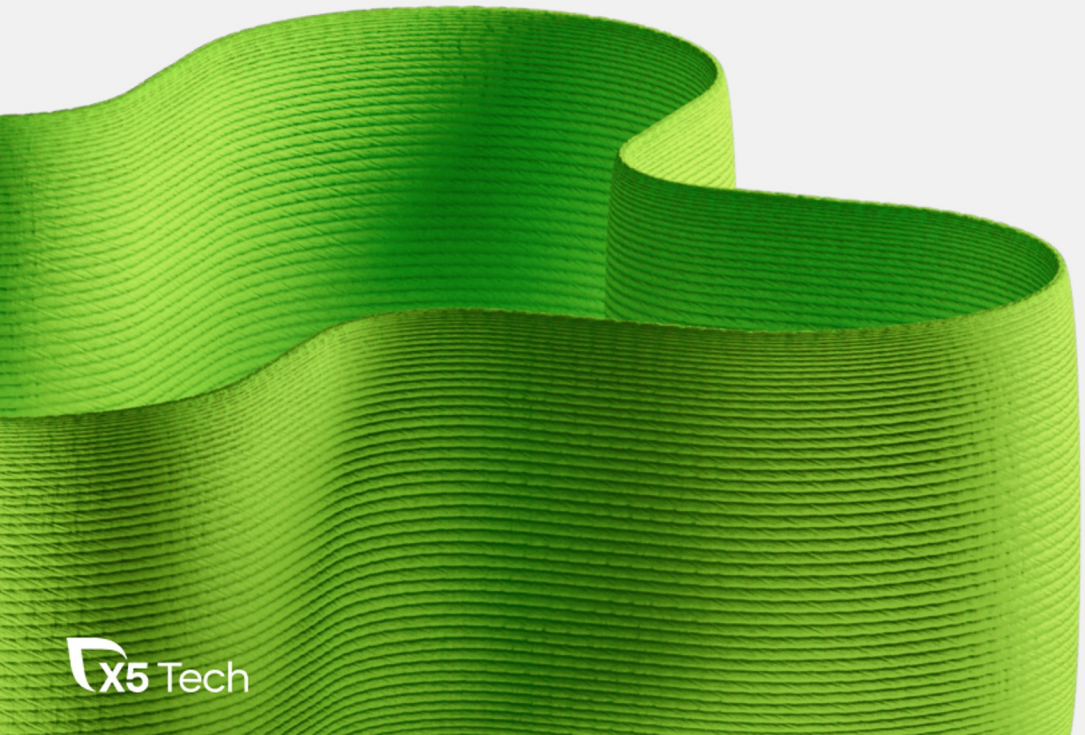
Сейчас применим этот путь к нескольким функциям

ЗАДАНИЕ ДЛЯ ПРАКТИКИ

Реализуйте алгоритмы на типах, циклах и ветвлениях

1. Написать FizzBuzz через switch
2. Найти min, max и сумму среза
3. Посчитать среднее без потери точности
4. Обработать пустой вход
5. Довести автотесты до PASS

Рефлексия



Удалось ли:

- Понять отличие var от :=
- Выбрать тип для денег
- Запомнить формы циклов
- Освоить конструкцию switch

Где это пригодится в работе?

Что осталось непонятным?

Домашнее задание

Задание: Закрепите типы, циклы и управляющие конструкции. Не меняя исходные сигнатуры и автотесты, решите представленные в репозитории задачи. Проверьте преобразования и границы

Формат сдачи: Ссылка на ветку в репозитории

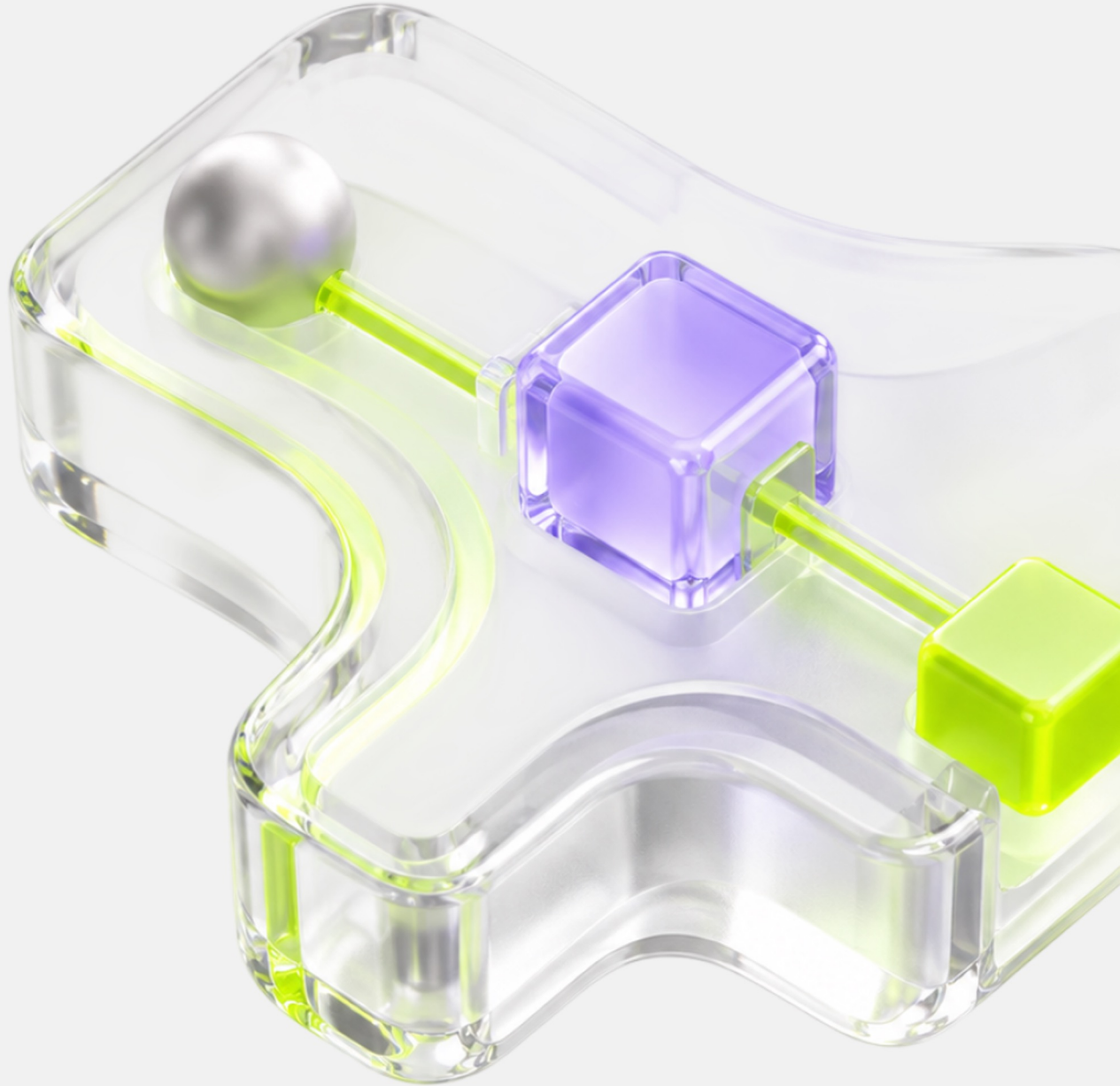
Критерии оценивания: Процент пройденных автотестов



ТЕМА СЛЕДУЮЩЕГО ЗАНЯТИЯ:

Функции, указатели и составные типы

1. Функции
2. Указатели
3. slice, map и struct
4. Разбор задания



Итоги занятия

1. Типы и управляющие конструкции задают правила алгоритма
2. Сначала решаем, как хранить данные – после этого алгоритм становится проще

ТЕПЕРЬ ВЫ УМЕЕТЕ:



Выбирать подходящий тип

Различать int, float64, string, bool, byte и rune



Объявлять данные

Использовать var, :=, const и zero values



Строить условия и циклы

Применять if, for, range и switch



Учитывать граничные значения

Замечать переполнение, точность и пустые значения

X5 Tech